

Hermes Adventure Game: Engine Architecture & Implementation Plan

This document outlines the architectural paradigm and implementation roadmap for the Hermes Adventure Game engine. It details the separation of concerns between the deterministic Python state engine and the LLM-driven narrative renderer, with a specific focus on solving multi-room puzzles and dynamic environmental states without combinatorial explosion.

1. Core Architectural Philosophy

The engine enforces a strict boundary between state management and narrative flavor to prevent LLM hallucination and state corruption.

- **Deterministic Backend (Python/SQLite):** The game.py CLI and engine.py modules own the absolute truth. They manage the 3D grid, inventory, player HP, edge connectivity, and validation rules. The LLM cannot mutate state directly; it can only propose changes via JSON.
- **Narrative Renderer & Referee (LLM):** The LLM acts as the Game Host. It reads the deterministic state at the start of every turn, adjudicates complex free-form actions based on physical plausibility, and synthesizes the raw data into cohesive prose.
- **Validation Layer (Pydantic):** All LLM proposals (room generation, state changes) must pass strict JSON schema validation.

2. Environmental States & "Composition over Hardcoding"

To avoid the exponential trap of writing 2^N room descriptions for every possible combination of environmental states (e.g., flooded, dark, toxic), the engine utilizes an entity-based composition model.

Concept	Implementation Directive
Base Geometry	The generated room description stored in SQLite must be strictly neutral and limited to permanent physical architecture (e.g., "A

Concept	Implementation Directive
	square concrete generator room with a sunken trench.").
Hostile States as Entities	Temporary or hostile environmental conditions (darkness, flooding) are generated as obstacle entities. They must possess is_blocking: true, a specific blocks_direction, and a solution_condition.
Narrative Synthesis	The LLM reads the base geometry and active obstacle entities, then seamlessly weaves them into a single descriptive paragraph for the player. When an obstacle is mechanically cleared, the LLM naturally drops it from future descriptions.

3. Multi-Room Puzzles via "Lazy Evaluation"

Puzzles that span multiple rooms (e.g., flipping a breaker in Room A to drain water in Room C) are resolved dynamically using global flags and LLM logic, avoiding the need for deep database mutations across unvisited coordinates.

1. **Flag Creation:** When the player solves a localized puzzle, the LLM proposes a flags_set payload (e.g., "power_restored": true) via the apply command.
2. **Context Injection:** The Python engine injects the global flags dictionary into the LLM's context during both regular turns (game.py state) and Just-In-Time generation (game.py create-room).
3. **Auto-Resolution on Entry:** When a player enters a room, the LLM checks if the global flags satisfy any of the room's environmental obstacles. If true, the LLM automatically calls apply to clear the obstacle and narratively renders the environment changing (e.g., "The restored power banishes the darkness as you step inside.").

4. Execution Plan for Coding Agent

The following technical updates must be implemented in the codebase.

Task A: Update Context Injection in engine.py

Modify the `_build_generation_context` function to supply global flags to the LLM during room generation.

```
def _build_generation_context(
    conn: sqlite3.Connection, current_node: sqlite3.Row, direction: str, tx: int, ty: int, tz: int
) -> dict:
    zone = database.get_zone_config(conn)
    player_row = database.get_player_state(conn) # 1. Fetch player state

    # ... [Keep existing neighbor calculation logic] ...

    return {
        "target_coordinates": {"x": tx, "y": ty, "z": tz},
        "direction_of_travel": direction,
        "exiting_room": {"name": current_node["name"], "description":
current_node["description"]},
        "zone": dict(zone) if zone else None,
        "neighbors": neighbors,
        "flags": json.loads(player_row["state_flags_json"]) # 2. Inject parsed flags
    }
```

Task B: Update SKILL.md Contract

Append the following instructions to the LLM's prompt directives:

Environmental States & Composition over Hardcoding

1. ****Base Geometry:**** When generating a room, the static `description` must only describe permanent physical architecture. Do not bake temporary states (like darkness, fire, or flooding) into the base description string.
2. ****Environmental Obstacles:**** Represent hostile environmental states (darkness, gas,

floods) as `obstacle` entities within the room. Set `is\_blocking: true`, assign a `blocks\_direction`, and define a clear `solution\_condition`.

3. ****Narrative Rendering:**** Synthesize the static base geometry and the active environmental obstacle entities into a cohesive, atmospheric description for the player.

4. ****Auto-Resolution on Entry:**** When reading a room's state, check if the player's global `flags` or inventory satisfy any of the room's environmental obstacles. If they do, call `game.py apply` to clear the obstacle immediately and narrate the environment changing (e.g., "The restored power flickers on, banishing the darkness.").

5. ****Setting Puzzle Flags:**** When a player solves a major mechanical puzzle, use `game.py apply` to submit a `flags\_set` payload (e.g., `{"power\_restored": true}`).